

Scan 3D des plats : solution retenue

Aujourd'hui, un restaurateur doit produire lui même un fichier .glb et le téléverser à la main dans Vorae. C'est le comportement prévu pour le MVP, mais ce n'est pas tenable dès qu'il faut équiper un menu entier. Ce document arbitre la façon dont les plats seront réellement capturés en 3D, et pourquoi.

■ Les quatre options évaluées

Option	Nature	Intégrable ?	Coût	Verdict
Meshy AI	Génération IA à partir d'une photo	Oui, API REST	~28 CA\$/mois	Écarté
Rig multi-caméras	Station physique et moteur IA	Indirect	Matériel à construire	Plus tard
RealityScan 2.1	Photogrammétrie de référence (Epic)	Oui, REST et gRPC headless	Gratuit sous 1 M\$ de revenu	Plan B
KIRI Engine API	Photogrammétrie managée	Oui, REST et webhooks	1 \$/scan, 10 gratuits, recharge min. 500 \$	Retenu

■ Pourquoi écarter la génération IA à partir d'une photo

Meshy et les moteurs équivalents ne capturent pas le plat : ils en inventent une interprétation. Les analyses techniques 2026 les situent explicitement sur le contenu stylisé de jeu vidéo, avec des arêtes au rendu pâte à modeler de près. Pour un menu où le convive doit reconnaître exactement ce qu'il va commander, l'écart entre le plat servi et le modèle affiché devient un risque commercial, pas un détail esthétique. Le test mené sur un burger l'a confirmé : géométrie approximative, et export bloqué derrière l'abonnement.

■ Pourquoi KIRI Engine plutôt que RealityScan

- **La photogrammétrie** de RealityScan, chez Epic, est le moteur le plus qualitatif du marché, et sa licence est gratuite sous 1 M\$ de revenu annuel. Mais son API suppose d'héberger et de maintenir notre propre machine GPU. Disproportionné pour un pilote à un seul restaurant.
- **KIRI Engine** expose une API REST managée : aucune infrastructure de notre côté, et la même photogrammétrie réelle, pas de la génération.
- **RealityScan** reste le plan B si la qualité ou le coût de KIRI ne tiennent pas à l'échelle. L'architecture en adaptateur décrite page 3 permet d'en changer sans réécrire le flux.

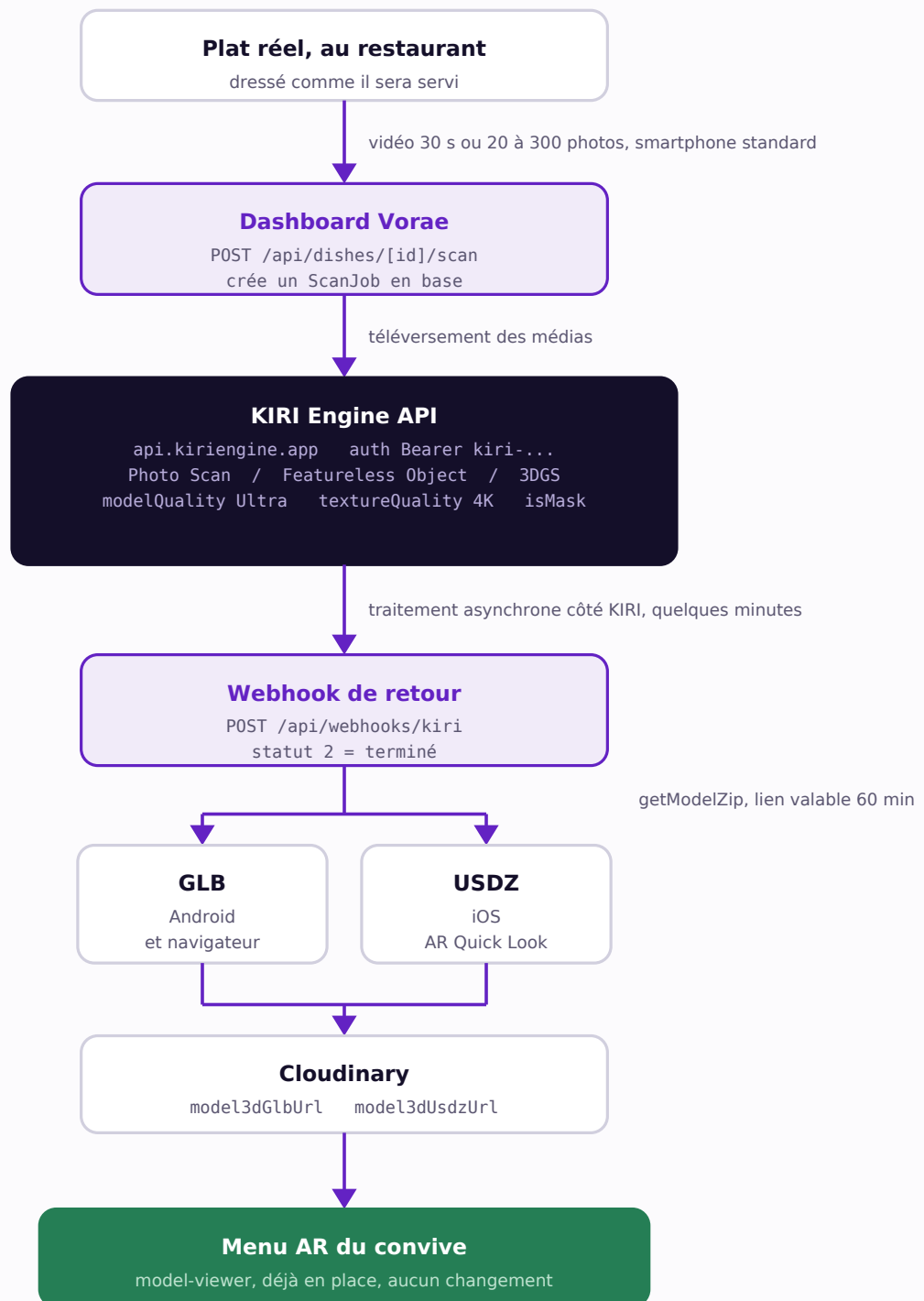
Trois détails qui font la différence pour de la nourriture

L'algorithme Featureless Object vise les surfaces brillantes et sans texture, exactement le cas des assiettes blanches et des sauces réfléchissantes où la photogrammétrie classique échoue. Le paramètre isMask isole le plat de son fond automatiquement. Et l'entrée accepte une vidéo de 30 secondes, pas seulement une série de photos : le restaurateur filme le tour de l'assiette, rien de plus.

■ Ce que la solution ne remplace pas

La section 15.3 du cahier des charges prévoit un service de capture professionnel facturé 45 \$ par plat, à forte marge. Ce flux automatisé ne s'y substitue pas : il le complète par le bas, en self-service, pour les restaurateurs qui veulent équiper un menu entier sans prendre rendez-vous. Les deux offres restent complémentaires.

Le flux, de l'assiette au menu AR



Les deux blocs violets sont les seuls à construire. Le bloc vert existe déjà et ne bouge pas : l'affichage AR de Vorae consomme exactement les formats que KIRI produit. Le retour par webhook est ce qui rend le flux compatible avec l'hébergement serverless de Vercel, qui ne peut pas tenir une connexion ouverte pendant plusieurs minutes.

- **Côté restaurateur** : il filme, il ne prend pas 200 photos et ne manipule aucun fichier 3D.
- **Côté formats** : les deux formats arrivent du même export, sans conversion maison.
- **Côté serveur** : KIRI rappelle Vorae quand c'est prêt, rien à interroger en boucle.

Intégration et suite

■ Ce qui existe déjà, ce qu'il faut ajouter

Élément	État	Détail
Affichage AR	Existe	ar-viewer.tsx, model-viewer et AR Quick Look. Aucun changement.
Champs du modèle	Existe	Dish.model3dGlbUrl et Dish.model3dUsdzUrl, déjà au schéma Prisma.
Stockage	Existe	Cloudinary en resource_type raw, déjà utilisé par l'upload manuel.
Commande du scan	À créer	POST /api/dishes/[id]/scan : reçoit la vidéo, appelle KIRI, ouvre un job.
Réception	À créer	POST /api/webhooks/kiri : télécharge le zip, extrait, téléverse, remplit les champs.
Traçabilité	À créer	Modèle Prisma ScanJob : statut, fournisseur, coût, message d'erreur.
Adaptateur	À créer	lib/scan3d.ts sur le modèle de lib/billing.ts, pour changer de fournisseur sans toucher aux routes.

■ Un effet de bord utile

La route d'upload actuelle porte ce commentaire : la conversion .glb vers .usdz est demandée par la section 9.2 du cahier, mais aucun outil fiable n'existe côté serveur Node. KIRI livre les deux formats dans le même export. La dette D-03 du tableau de bord disparaît sans travail supplémentaire.

■ Garde-fous à prévoir

Chaque scan coûte de l'argent réel, contrairement à l'upload manuel d'un fichier. Il faut donc un compteur et une limite par restaurant dès la première version, jamais un accès illimité non facturé, pour éviter qu'un usage abusif ne génère une facture fournisseur incontrôlée. Deux pistes : un quota inclus par palier d'abonnement Stripe, ou un nouveau packageType sur le modèle DishCaptureOrder existant.

■ Coût réel, confirmé

- **Tarif** : 1 appel API = 1 crédit = 1 \$ US, identique pour Photo Scan, Featureless Object et 3DGS. Pas de palier caché selon la qualité demandée.
- **Essai gratuit : 10 crédits**, suffisants pour tester la qualité sur de vrais plats sans rien payer, avant tout engagement.
- **Recharge** : 500 crédits (500 \$ US) minimum au-delà du gratuit. C'est une décision de budget à prendre avant de dépasser 10 plats, pas un détail technique à vérifier.
- **La qualité reste à juger** sur de vrais plats du restaurant pilote, pas sur des photos de stock. La nourriture reste le cas difficile de la photogrammétrie : surfaces molles, sauces, garnitures.

Prochaine étape concrète

Créer le compte KIRI Engine et transmettre la clé API. Le squelette d'intégration (routes, modèle ScanJob, adaptateur) peut être construit en parallèle, et se testera immédiatement avec les 10 crédits offerts, sur un plat réel filmé au téléphone. La décision de recharger 500 \$ ne se prend qu'après ce test, pas avant.